

Improving Track Times Using Dynamic Pure Pursuit Controllers

Akhil Kothapalli
VIP

Georgia Institute of Technology
akothapalli31@gatech.edu

Abstract—This paper presents my work on improving autonomous F1TENTH race car performance through improved Pure Pursuit based controllers. Experiments were conducted on F1TENTH tracks (approximately 20-meter loops) containing smooth bends and long straightaways. The baseline is a static pure pursuit controller with fixed look-ahead and constant velocity. Two new controllers are evaluated, a reactive dynamic pure pursuit controller that adjusts look-ahead distance and velocity in real time based on local curvature, and an offline adaptive controller that pre-computes look-ahead distances and velocity from the planned raceline. The offline controller is the primary contribution of this work. Experiments on several tracks show that it significantly reduces lap times, achieving up to a 62.1% improvement relative to the static baseline.

I. INTRODUCTION

This work was conducted within Georgia Tech’s Vertically Integrated Projects (VIP) program on Active Safety for Autonomous and Semi-Autonomous Vehicles. Each semester builds on prior ones working towards developing a fully autonomous racing pipeline. During the first semester, the focus was on implementing reactive controllers such as Follow the Gap and the Disparity Extender. These algorithms use LIDAR to find a point with the greatest distance (essentially the largest gap) and steer toward it. Since the car has no memory of previously seen parts of the track, it must compute the best direction on the fly, hence the term “reactive algorithm.” Although these methods can perform well, they fail on more complex tracks (as shown in Fig. 1), where full mapping and trajectory optimization provide a much greater advantage. With a mapped environment, the vehicle can plan for every turn ahead of time instead of reacting only when something appears.

During the second semester, we focused on achieving reliable mapping and localization using SLAM Toolbox and then pushing for improved lap times. In practice, getting mapping and localization to work consistently proved to be very challenging and consumed most of the semester. By the end, we developed a reasonably reliable SLAM and localization pipeline. From here we were able to use the VipPathOptimization package to generate racelines from the occupancy grid created by SLAM toolbox. Due to the limited time, we were only able to make a suboptimal Pure Pursuit controller which later proved to be a bottleneck. (<https://youtu.be/ruCTsLBYeas>).

Active Safety for Autonomous and Semi-Autonomous Vehicles

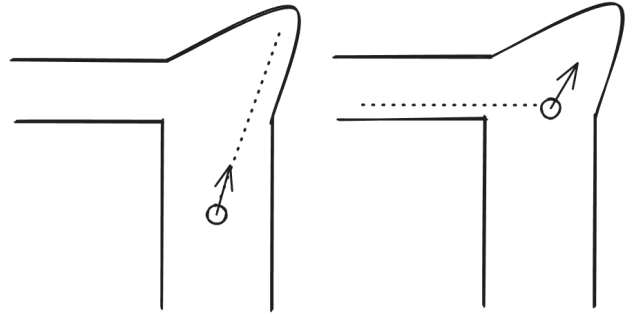


Fig. 1. This figure shows the main issue with Follow the Gap. The arrow shows the car’s velocity, and the dotted line is the farthest gap that it follows. In the left image, the car starts turning toward the biggest gap. In the right image, it figures out the correct direction too late and ends up hitting the wall.

II. PROBLEM STATEMENT

In the past we have only looked towards reactive algorithms and recently started making developments in mapping and localization. In order to reach competitive numbers, we need to improve all aspects of the baseline. One area that can be greatly improved is the controller or how the car decides to adjust the steering and velocity based on the points it must follow. We have tried a simple implementation of pure pursuit in the past, but have not reached the limits of this work. The goal of this paper is to improve lap times significantly by implementing an efficient pure-pursuit controller. This would also help others build on the work, enabling them to explore new ideas such as static and dynamic vehicle avoidance, or improve localization, mapping and trajectory optimization further by using a reliable and fast controller.

III. RELATED WORK

This work takes inspiration from the Regulated Pure Pursuit (RPP) controller proposed by Macenski, Martin, Singh, and Gines [1]. RPP extends the traditional Pure Pursuit algorithm by dynamically adjusting velocity based on both path curvature and obstacle proximity. This ensures safer navigation in constrained environments, such as facility aisles, where maintaining full speed is not acceptable. The controller developed in this study closely aligns with these principles, focusing on curvature-based dynamic control.

Regulated Pure Pursuit, like other Pure Pursuit variants, does not model vehicle dynamics and relies solely on geometric path information. As a result, it cannot account for dynamic constraints while adjusting velocity.

Another reference for this controller's design is the work of Sukhil and Behl on Adaptive Look-Ahead Pure Pursuit for autonomous racing [2]. Their paper introduces an offline algorithm that assigns an optimal look-ahead distance to every waypoint on a racing line. Instead of using a fixed look-ahead value, their method evaluates multiple candidates at each waypoint using three tunable objectives: maximizing exit velocity, minimizing deviation from the reference trajectory, or a convex combination of both.

Their results show that adaptive look-ahead significantly outperforms standard Ackermann-adjusted Pure Pursuit. In simulation and on the F1/10 real testbed, their adaptive controller achieved up to a 20% improvement in lap time and higher average speeds.

We implement their work as it directly applies to F1TENTH and takes advantage of the fact that we already have a map set up and can pre-compute the look-ahead. It also takes both trajectory error and speed into account. The drawback is the computational time it takes and that it introduces more parameters to tune.

IV. METHODOLOGY

A. Reactive Pure Pursuit

This controller computes the look-ahead distance and target velocity reactively based on the curvature of the path ahead. At each step, it estimates curvature from the waypoints, removes small variations with a dead zone, and normalizes the result. Higher curvature shortens the look-ahead and lowers speed, while straighter segments increase both. The nonlinear scaling (square root for look-ahead and cubic for velocity) makes the controller stable in turns and fast on straightaways.

The dynamic controller computes curvature κ at each waypoint using finite difference approximations of the path derivatives:

$$\kappa_i = \frac{x'_i y''_i - y'_i x''_i}{(x'^2_i + y'^2_i)^{3/2}}$$

A curvature dead zone filters out slight turns:

$$\kappa_{\text{eff}} = \max(0, |\kappa| - \kappa_{\text{deadzone}})$$

where $\kappa_{\text{deadzone}} = 0.5$ treats gentle curves as straight segments.

The effective curvature is normalized to $[0, 1]$ and used to compute dynamic look-ahead distance and velocity:

$$\kappa_{\text{norm}} = \frac{\min(\kappa_{\text{eff}}, \kappa_{\text{max}})}{\kappa_{\text{max}}}, \quad L_d = L_{\min} + (L_{\max} - L_{\min})(1 - \kappa_{\text{norm}})^{0.5}$$

$$v_{\text{target}} = v_{\min} + (v_{\max} - v_{\min})(1 - \kappa_{\text{norm}})^3$$

The cubic speed scaling $(1 - \kappa_{\text{norm}})^3$ maintains high speeds on straightaways while aggressively reducing speed only in tight turns. The square root look-ahead scaling $(1 - \kappa_{\text{norm}})^{0.5}$ provides less aggressive reduction to increase stability.

B. Offline Adaptive Pure Pursuit

This pure pursuit controller will compute lookahead and velocity offline. The VipPathOptimization [3] repository contains code to take in an occupancy grid and generate race lines. We can use this to produce points on the raceline with the following fields:

- 1) (x, y) : the position of the car along the planned path (previously used in the original controller)
- 2) heading: the direction the vehicle should be facing
- 3) velocity: the target velocity at each (x, y) coordinate
- 4) curvature: the curvature of the race line at each point
- 5) left and right boundaries: the distance to the track walls relative to the current position

With these values, the controller no longer needs to compute velocity reactively. Instead, it can attempt to follow the planner's theoretical velocity profile directly. Additionally, using the boundary information, the likelihood of collisions can be greatly reduced by shutting down or slowing the vehicle when its orientation indicates it is heading toward a wall.

In addition to the controller, I implemented the Adaptive Look-Ahead method proposed by Sukhil and Behl [2]. Each point's look-ahead is evaluated using:

- 1) Maximum Velocity Objective:

$$\pi_v = \arg \max_L \left(\sum_{i=1}^N v_{\text{exit}, i} \right)$$

This selects the look-ahead distance that maximizes the total exit velocity across all waypoints.

- 2) Minimum Deviation Objective:

$$\pi_\delta = \arg \min_L \left(\sum_{i=1}^N \delta_i \right)$$

Here, deviation is computed as the area between the driven trajectory and the reference path.

- 3) Convex Combination Objective:

$$\pi_{v-\delta} = \beta \pi_v + (1 - \beta) \pi_\delta$$

The parameter $\beta \in [0, 1]$ controls the balance between maximizing speed and minimizing deviation.

In practice, tuning β depends heavily on track geometry. Tracks with tight curves require lower β to maintain precision, while tracks with long straight sections benefit from higher β to maintain speed.

V. RESULTS

For all results below, here are the parameters I set for each controller:

- Static Pure Pursuit (Baseline)
 - Look-ahead distance: 1.0 m
 - Speed: 0.65 m/s
- Reactive Pure Pursuit
 - $L_{\min} = 0.3$ m, $L_{\max} = 1.5$ m
 - $v_{\min} = 0.3$ m/s, $v_{\max} = 1.0$ m/s

- $\kappa_{\max} = 5.0$
- Offline Adaptive Pure Pursuit
 - Velocity scaling factor: 0.5
 - Adaptive parameter: $\beta = 0.5$
 - Lookaheads evaluated: [0.5, 0.65, 1.0, 2.0]

These parameters were selected after testing for stability and speed.

A. Baseline Static Pure Pursuit vs Reactive Pure Pursuit

To compare against the previous static pure-pursuit controller, two test runs were conducted.

- Static Pure Pursuit (Baseline): 16.27-second lap time
<https://youtu.be/AVKHX7uBw6k>
- Reactive Pure Pursuit: 11.87-second lap time
<https://youtu.be/Hlp1JhpXb7U>

This represents a 27.04% reduction in lap time, from the first Dynamic Pure Pursuit controller.

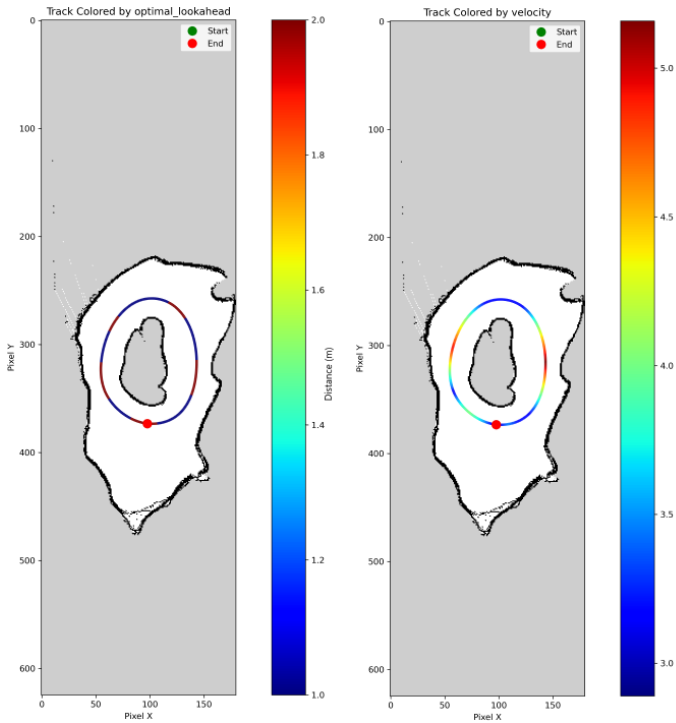


Fig. 2. Shows the calculated lookahead distance and velocities and various points along the optimized raceline for the circular track

B. Baseline vs Offline Adaptive Pure Pursuit on Circular Tracks

- Static Pure Pursuit (Baseline): 15.41-second lap time
<https://youtu.be/Lbv2fS6zmjo>
- Offline Adaptive Pure Pursuit: 7.24-second lap time
https://youtu.be/JFAp9mq-_sU

In the videos, you can see that the track has a fair amount of width. This is due to the baseline struggling to stay within bounds.

Fig. 2 shows the optimal look-ahead and velocity for the particular track. You can also notice that this closely matches the video. With this controller, we were able to reach a 53.02% reduction in lap time, from the baseline controller.

C. Baseline vs Offline Adaptive Pure Pursuit on Straightaways

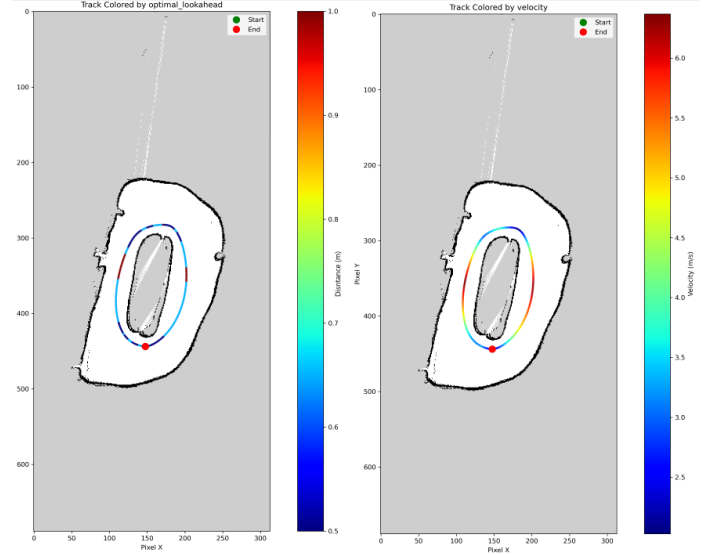


Fig. 3. Shows the calculated look-ahead distance and velocities and various points along the optimized trajectory for the track with straightaways

We also need to evaluate performance on straightaways. The calculated velocity and look-ahead are in Fig. 3:

- Static Pure Pursuit (Baseline): 21.8-second lap time
https://youtu.be/Eo1Fud_wKqs
- Offline Adaptive Pure Pursuit: 8.25-second lap time
<https://youtu.be/F9VmR2KpyuU>

This produced a 62.1% reduction in lap time.

D. Dynamic Pure Pursuit using Trajectory Planner Values on mixed track

In Fig. 4, we have a track with a smaller width and more of a straightaway. The parameters were also set more aggressively (higher β and velocity scaling factor). The baseline was unable to run without hitting the walls, so we do not have that for comparison. The lap time was 6.83 seconds. <https://youtu.be/7GKr3Wpc2RY>

VI. DISCUSSION

As shown in the results, both controllers managed to outperform the baseline. However, looking at this alone is not sufficient, a more thorough evaluation is necessary.

A. Static Pure Pursuit (Baseline)

This is a very simple implementation where we tested a series of look-aheads and speeds, and the configuration presented in results produced the most reliable (non-crashing) results. The downside is that it is very slow and does not take advantage of already having the map beforehand. Nonetheless, it serves as a strong baseline.

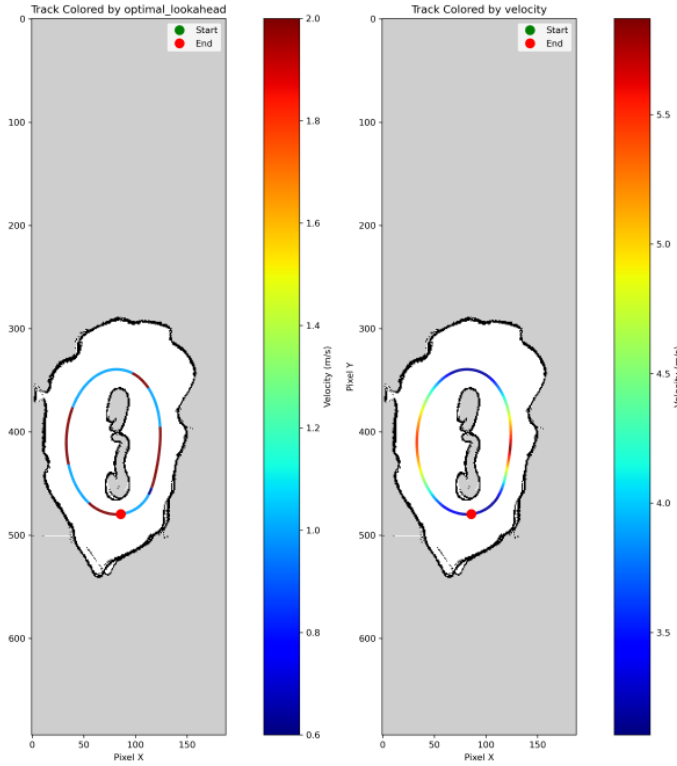


Fig. 4. Shows the calculated lookahead distance and velocities and various points along the optimized trajectory for the track with straightaways

TABLE I

LAP TIME RESULTS FOR BASELINE, REACTIVE PP, AND OFFLINE LA PP

Track Type	Lap Time Comparison		
	Controller	Lap Time (s)	Improvement
Test Run	Baseline (Static PP)	16.27	–
Test Run	Reactive PP	11.87	27.04%
Circular Track	Baseline (Static PP)	15.41	–
Circular Track	Offline LA PP	7.24	53.02%
Straightaway	Baseline (Static PP)	21.80	–
Straightaway	Offline LA PP	8.25	62.1%
Mixed Track	Baseline (Static PP)	DNF	–
Mixed Track	Offline LA PP	6.83	–

^aPP = Pure Pursuit, LA = Look-ahead.

B. Reactive Pure Pursuit

This controller uses the coordinates from the generated raceline to compute curvature at each point. Based on this curvature, it adjusts both the velocity and the look-ahead distance. These values are computed reactively as the car moves along the track. This controller was able to reduce lap times by 27.04% on a relatively simple track.

A noticeable issue with this controller is that the car's laps are not consistent, the path varies each time. This becomes problematic at higher speeds, where even small deviations can cause the car to drift into the walls. Another issue is the behavior of the speed controller, where it appears as if the car only has two speeds, even though the controller technically allows for a full range. In practice, the curvature penalty is so

high that the car drops to the lowest speed on turns and jumps to the highest speed on anything resembling a straightaway.

Overall, this method does not take advantage of already knowing the map. It inherits many of the drawbacks of a reactive algorithms.

C. Offline Adaptive Pure Pursuit

This controller computes look-ahead distances and velocities offline before the car is ran. It evaluates multiple look-ahead candidates at each waypoint and selects the one that best minimizes deviation and maximizes exit velocity. This controller outperformed the baseline on both circular and straight tracks, achieving lap time reductions of 53.02% and 62.1%, respectively.

One issue observed on straightaways leading into tight bends is that the car often hits the wall. This occurs because the velocity remains high as we exit the straightaway while the look-ahead becomes small, as to minimize trajectory error. The combination of high speed and short look-ahead results in unpredictable behavior and frequent crashes. Future work could address this by integrating the adaptive look-ahead directly with the trajectory planner rather than treating them as two separate steps, allowing them to work together to compute more reliable values.

Another issue is that although the look-ahead is controlled well, several additional parameters require tuning. We introduce β and a set of candidate look-ahead values, leaving the selection to the user, who may not identify the optimal configuration.

D. Future Work

- Tuning the Offline Adaptive Pure Pursuit's parameters across many tracks
- Combining adaptive look-ahead calculations with trajectory optimization
- Other controllers like MPC

REFERENCES

- [1] S. Macenski, F. Martin, S. Singh, J. Gines, "Regulated Pure Pursuit for Robot Path Tracking," *arXiv preprint arXiv:2305.20026*, 2023.
- [2] V. Sukhil and M. Behl, "Adaptive look-ahead Pure-Pursuit for Autonomous Racing," *arXiv preprint arXiv:2111.08873*, 2021.
- [3] N. Zhang, "VipPathOptimization," Feb. 26, 2025. <https://github.gatech.edu/AE-VIP/VipPathOptimization> (accessed Nov. 19, 2025).